

مستند فنی کامل پروژه bichart

معماری، جریان اجرا، API ها، مدل داده، تمام فایل های موجود در پروژه و
منطق کدهای اصلی — به فارسی

تعداد فایل های بررسی شده

123 فایل

برنج فعلی

seperateSQLEdition

تغییرات ثبت نشده Git

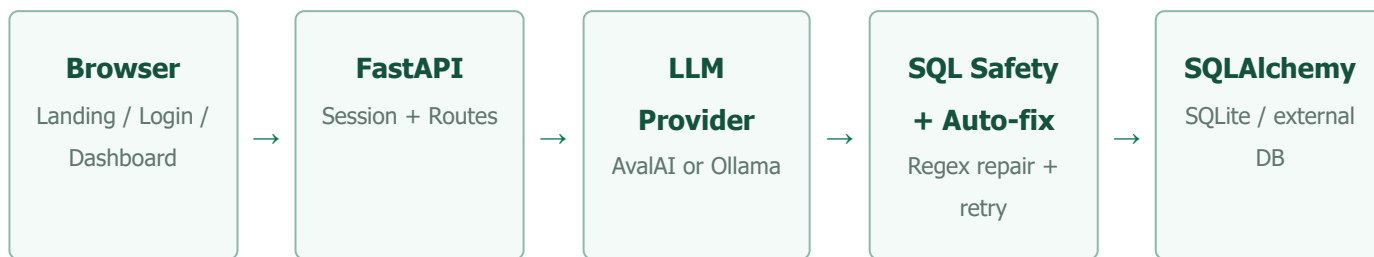
93 مورد

تولید سند

2026-07-27 18:00

این سند مستقیماً از فایل های working tree خوانده شده است (نه از یک commit گیت خاص)، پس شامل تمام تغییرات همین نشست کاری هم می شود. مقادیر secret عمداً در سند بازنشر نشده اند.

خلاصه سامانه



مسیر اصلی سؤال: مرورگر سؤال و تنظیمات زبان/مدل را به `/api/chat` می‌فرستد؛ مدل یک JSON شامل SQL و توضیح می‌سازد. کاربر SQL را (در صورت نیاز) ویرایش و اجرا می‌کند: `/api/run_sql` ابتدا SQL را نرمال‌سازی و ایمن‌سازی می‌کند، سپس اجرا می‌کند؛ اگر اجرا با خطا مواجه شود، خطای دیتابیس یک‌بار به‌صورت خودکار به مدل بازخورد داده می‌شود تا SQL را اصلاح کند (`fix_sql`) و دوباره اجرا شود. نتیجه، زمان هر مرحله و (در صورت اصلاح خودکار) پرچم `auto_fixed` به رابط برمی‌گردد. تحلیل هوشمند عمداً در درخواست جداگانه `/api/analyze` و با اقدام کاربر اجرا می‌شود تا مصرف مدل کنترل‌شده بماند.

- **هدف:** پرسش فارسی/انگلیسی درباره تراکنش‌های پرداخت، تبدیل به SQL و نمایش جدول/نمودار/تحلیل.
- **Backend:** FastAPI، Pydantic، SQLAlchemy و Pandas.
- **مدل زبانی:** AvalAI (ابری، پیش‌فرض) یا Ollama (محلی) با interface یکسان `.generate_sql/generate_analysis/generate_discussion/fix_sql`.
- **Frontend:** HTML/CSS/JavaScript خالص (بدون framework) و Chart.js.
- **هویت:** session امضا شده، حساب `app_users` و نقش `admin/user`.
- **داده:** star schema آزمایشی PSP روی SQLite با قابلیت تغییر `DATABASE_URL` به DB2/SQL Server/PostgreSQL.

قابلیت‌های تازه این نشست کاری

۱. پنل سؤالات پرتکرار سراسری

`MemoryManager.get_frequent_questions` + مسیر `GET /api/frequent-questions` + کارت جدید در `sidebar`.
سؤالات میان همه کاربران شمارش و ۱۰ مورد برتر (بر اساس تکرار، سپس تازگی) نمایش داده می‌شود؛ کلیک روی هرکدام همان لحظه پرسیده می‌شود.

۲. بازپخش کامل تاریخچه

`run_sql` اکنون نمونه واقعی ردیف‌های نتیجه را نیز در `history.json` ذخیره می‌کند (`HISTORY_DATA_CAP=500`). کلیک روی یک آیتم تاریخچه، سؤال/SQL/جدول/نمودار/تحلیل/وضعیت بازخورد همان نوبت قدیمی را دوباره می‌سازد.

۳. بازنگری نمودارها + چاپ

باگ اصلی: سه `select` در چارت همگی کلاس `chart-type-select` داشتند و `querySelector` عنصر اشتباه (ستون برچسب) را به جای منوی نوع نمودار انتخاب می‌کرد — با کلاس یکتای `chart-kind-select` رفع شد. علاوه بر آن: مقدار نادرست `tooltip` در حالت افقی، نمایش نادرست نمودار دایره‌ای با مقادیر منفی، و نبود به‌روزرسانی یادداشت «نمایش N از M» پس از تعویض ستون اصلاح شدند؛ نوع پیش‌فرض همیشه ستونی (عمودی) شد و دکمه «چاپ نمودار» (خروجی PNG در پنجره چاپ) اضافه شد.

۴. بازطراحی کارت‌های نتیجه + آکاردئون بحث

هر بخش (SQL، جدول، نمودار، تحلیل، بحث، بازخورد) اکنون یک کارت رنگی مجزا با نوار رنگی بالا (`--section-` `accent`) است، نه یک نوار خطچین پیوسته. بخش «بحث و بررسی» به یک آکاردئون جمع‌شونده تبدیل شد.

۵. حلقه خوداصلاحی اجرای SQL

تابع تازه `fix_sql` در هر دو کلاینت مدل (با `prompt` مشترک `_fix_sql_prompt`) و منطق تازه در `run_sql` : اگر اجرای SQL خطا بدهد، خطای واقعی دیتابیس یکبار به مدل بازخورد داده می‌شود تا SQL را اصلاح کند و دوباره اجرا شود؛ در صورت موفقیت پرچم `auto_fixed` برگردانده و یک یادداشت سبز به کاربر نشان داده می‌شود.

۶۰ اصلاح لینک لینکدین

لینک نماد لینکدین در فوتر `static/first.html` که placeholder بود، به آدرس واقعی صفحه شرکت اصلاح شد.

مسیرهای HTTP و سطح دسترسی

مسیر	دسترسی	وظیفه
GET /	عمومی	صفحه معرفی محصول <code>static/first.html</code> .
GET /login	عمومی	فرم ورود؛ کاربر واردشده مستقیم به <code>/app</code> هدایت می‌شود.
POST /api/login	عمومی	بررسی کپچا، احراز هویت با <code>users.py</code> و ساخت نشست.
POST /api/logout	واردشده	پاک کردن کامل نشست.
GET /api/me	واردشده	اطلاعات کاربر جاری + ترجیحات (<code>provider/language</code>).
GET /favicon.ico	عمومی	آیکون سایت.
GET /app	واردشده	پوسته داشبورد گفتگو (<code>static/index.html</code>).
GET /users	مدیر	صفحه مدیریت کاربران؛ غیرمدیر به <code>app/</code> هدایت می‌شود.
GET /api/users	مدیر	فهرست کامل حساب‌های برنامه.
POST /api/users	مدیر	ساخت حساب جدید و هش کردن گذرواژه.
PUT /api/users/{id}	مدیر	ویرایش نام کاربری، رمز، نقش، وضعیت فعال.
DELETE /api/users/{id}	مدیر	حذف حساب؛ حذف خود و آخرین مدیر مسدود است.
GET /api/health	واردشده	بررسی سلامت Ollama، DB، AvalAI.
GET /api/avalai/credit	واردشده	پروکسی سرور برای مانده اعتبار AvalAI (کلید هرگز به مرورگر نمی‌رود).
POST /api/chat	واردشده	گام ۱: تولید SQL از سؤال کاربر (بدون اجرا).
POST /api/run_sql	واردشده	گام ۲: اعتبارسنجی ایمنی، اجرا؛ در صورت خطا یک تلاش خودکار اصلاح SQL توسط مدل.
POST /api/analyze	واردشده	تحلیل اختیاری حداکثر ۸۰ ردیف اول نتیجه.
POST /api/discuss	واردشده	بحث/چالش چندنوبتی درباره داده یا تحلیل.
POST /api/feedback	واردشده	ثبت 🍌 / 🍌 روی یک پیام برای یادگیری از بازخورد.
GET /api/history	واردشده	۵۰ پیام اخیر همان کاربر، شامل داده ذخیره شده برای بازپخش.

مسیر	دسترسی	وظیفه
GET /api/frequent-questions	وارد شده	۱۰ سؤال پرتکرار از میان همه کاربران (جدید).
GET /api/preferences	وارد شده	خواندن provider/language ذخیره شده کاربر.
PUT /api/preferences	وارد شده	ذخیره provider/language معتبر.
POST /api/export_pdf	وارد شده	ساخت و بازگرداندن PDF فارسی نتیجه (fpdf2 + HarfBuzz).
POST /api/export_excel	وارد شده	ساخت و بازگرداندن اکسل سه شیتی (Data/Explanation/Analysis).
GET /api/feature-poll	وارد شده	وضعیت نظرسنجی قابلیت های آینده.
POST /api/feature-poll	وارد شده	ثبت رأی کاربر (حداکثر ۳ گزینه + پیشنهاد سفارشی).

مدل داده

جدول	نقش	ستون‌ها/رابطه‌های مهم
app_users	حساب‌های ورود	id, username, password_hash (PBKDF2), mobile, display_name, role, is_active
dim_date	بعد تاریخ	date_key به شکل عدد YYYYMMDD, full_date, روز/ماه/سال, آخر هفته و تعطیلی
dim_category	بعد صنف	شناسه و عنوان فارسی ۱۵ صنف استاندارد پذیرندگان
dim_customer	مشتری قراردادی PSP	نام, access_key نمایشی و نقش customer/admin; مستقل از app_users است
dim_merchant	پذیرنده	نام/کد, MCC, صنف, شهر, مالک (owner_customer_id) و وضعیت فعال
dim_terminal	پایانه	سریال, پذیرنده, نوع POS/mPOS/Online, تاریخ نصب و وضعیت active/inactive
fact_transactions	فکت تراکنش	تاریخ, پایانه, پذیرنده, مبلغ, وضعیت PAN, approved/declined/reversed ماسک‌شده, کانال

قاعده حیاتی تاریخ: fact_transactions.date_key یک عدد YYYYMMDD است؛ برای هر فیلتر تقویمی باید dim_date جوین شود و از full_date استفاده گردد. هم prompt مدل (app.py در SCHEMA_DESCRIPTION) و هم sql_safety.py این قاعده را تقویت/تعمیر می‌کنند.

فایل‌های Backend (پایتون)

مرکز اتصال تمام اجزای برنامه: تعریف middleware، FastAPI ها، مدل‌های Pydantic و تمام endpoint ها.

- دو provider با مجموعه VALID_PROVIDERS کنترل می‌شوند و `get_llm_client` ماژول AvalAI یا Ollama را برمی‌گرداند.
- ترتیب middleware مهم است: `SessionMiddleware` بعد از تعریف middleware ورود اضافه می‌شود تا در پشته بیرونی‌تر باشد و `request.session` از قبل در دسترس middleware ورود باشد.
- رشته `SCHEMA_DESCRIPTION` ساختار دقیق جداول و قواعد حیاتی تاریخ/وضعیت را به مدل می‌دهد تا SQL سازگار با SQLite تولید شود.
- `/api/chat` فقط SQL تولید می‌کند (بدون اجرا)؛ `/api/run_sql` جداگانه اعتبارسنجی و اجرا می‌کند تا کاربر بتواند پیش از اجرا SQL را ببیند/ویرایش کند.
- در `/api/run_sql` اگر اجرای SQL خطا بدهد، یک تلاش خودکار اصلاح انجام می‌شود: خطای واقعی دیتابیس به `llm.fix_sql(...)` داده می‌شود، خروجی دوباره از فیلتر `normalize/strip_limit/validate` عبور می‌کند و اجرا می‌شود؛ در صورت شکست اصلاح، خطای اصلی (نه خطای تلاش دوم) به کاربر برمی‌گردد.
- هر پیام موفق با `memory.add_message(..., data=records[:HISTORY_DATA_CAP])` ذخیره می‌شود؛ ذخیره خود داده (حداکثر ۵۰۰ ردیف) امکان «بازپخش کامل» یک گفتگوی قدیمی از تاریخچه را فراهم می‌کند.
- `/api/frequent-questions` سؤالات پرتکرار میان همه کاربران را از `memory.get_frequent_questions` می‌گیرد.
- خروجی PDF/Excel با `pdf_generator.generate_llm_pdf` و `excel_generator.generate_llm_excel` ساخته و به‌عنوان فایل داندلودی بازگردانده می‌شود.
- نظرسنجی قابلیت‌های آینده (feature poll) در همین فایل با قفل نخی و ذخیره در `feature_poll.json` پیاده شده است؛ گزینه‌های سفارشی کاربر تا ۱۰۰ کاراکتر مجاز است.
- مدیریت کاربران فقط با نقش admin در دسترس است؛ حذف خود کاربر جاری در همین route نیز مسدود می‌شود.

`get_llm_client(provider)` انتخاب پیاده‌سازی مشترک `generate_sql/generate_analysis/fix_sql`.

`require_login_middleware` تفکیک مسیرهای عمومی، API بدون نشست و صفحات نیازمند ورود.

`_startup_init_users` ساخت جدول `app_users` و مدیر آغازین هنگام `startup`.

`require_admin` تولید پاسخ 401 یا 403 برای عملیات مدیریتی.

`chat` گام تولید SQL؛ بدون اجرا.

`run_sql` اعتبارسنجی، اجرا، تلاش خودکار اصلاح در صورت خطا، ذخیره تاریخچه.

تحلیل ثانویه و بحث چندنوبتی روی نتیجه.

analyze / discuss

تاریخچه شخصی و سؤالات پرتکرار سراسری.

get_history /
get_frequent_questions

تولید فایل‌های خروجی از آخرین نتیجه.

export_pdf / export_excel

نظرسنجی قابلیت‌های آینده.

get_feature_poll /
post_feature_poll

خط 52

database.py

تنظیم مرکزی SQLAlchemy؛ تنها نقطه انتخاب SQLite یا پایگاه داده خارجی.

- DATABASE_URL از environment خوانده می‌شود و در حالت پیش‌فرض به `psp_bi_mock.db` اشاره دارد.
- برای SQLite گزینه `check_same_thread=False` فعال است؛ برای DB2/SQL Server/PostgreSQL فقط رشته اتصال عوض می‌شود.
- `pool_pre_ping=True` اتصال مرده را پیش از استفاده تشخیص می‌دهد.

موتور مشترک SQLAlchemy.

engine

کارخانه نشست با autocommit/autoflush خاموش.

SessionLocal

پایه declarative مدل‌های ORM.

Base

dependency مولد که نشست را در finally می‌بندد.

get_db

تعریف star schema آزمایشی و ساخت داده واقع‌گرایانه تراکنش‌های پرداخت ایران.

- شش کلاس ORM (ابعاد + فکت)، ۱۵ صنف استاندارد و MCC‌های متناظر، ~۵۵ پذیرنده فارسی، ۱۰ شهر و ۶ مشتری نمونه تعریف می‌کند.
- بازه تاریخ به صورت متحرک است: از ۶ ماه قبل تا امروز (`MOCK_DATE_START / MOCK_DATE_END`) — یعنی هر بار اجرا داده تازه‌ای متناسب با تاریخ جاری می‌سازد.
- seed ثابت ۴۲ باعث بازتولیدپذیری بخش تصادفی داده می‌شود.
- مبلغ تراکنش‌ها با `lognormvariate` تولید می‌شود تا توزیع واقعی‌تر (چوله به راست) داشته باشد.
- اجرای دوباره این فایل مخرب است: فایل دیتابیس را حذف یا (در صورت قفل بودن) جداول را drop/create مجدد می‌کند.

ابعاد تقویم و صنف.	<code>DimDate / DimCategory</code>
مالکیت تجاری، پذیرنده و پایانه.	<code>DimCustomer / DimMerchant / DimTerminal</code>
جدول رخدادهای مالی.	<code>FactTransaction</code>
انتخاب احتمالی وزن‌دار برای داده طبیعی‌تر.	<code>weighted_choice</code>
تولید شماره کارت ماسک‌شده نمایشی.	<code>mask_pan</code>
محاسبه تاریخ N ماه قبل با کلمپ روز پایان ماه.	<code>months_ago</code>
ساخت کامل schema و داده نمونه.	<code>build_database(n_transactions=1000)</code>

توابع کوچک و متمرکز برای احراز هویت مبتنی بر session امضا شده (Starlette SessionMiddleware).

- کلیدهای `username`، `user_id`، `role` و `display_name` را در کوکی نشست امضا شده نگه می‌دارد؛ خودِ کوکی رمزنگاری محتوا نمی‌کند، فقط امضا می‌شود.
- اعتبارسنجی واقعی گذرواژه به `users.py` واگذار می‌شود؛ این فایل فقط لایه نشست است.
- `SESSION_SECRET` قابل تنظیم با `environment` است؛ در نبود آن یک مقدار پیش‌فرض توسعه استفاده می‌شود که برای `production` باید حتماً عوض شود.

<code>verify_credentials</code>	روی <code>users.authenticate_user</code> wrapper
---------------------------------	--

<code>set_session_user</code>	نوشتن داده عمومی کاربر در نشست.
-------------------------------	---------------------------------

<code>is_authenticated / is_admin</code>	کنترل سریع وضعیت و نقش.
--	-------------------------

<code>current_user</code>	ساخت dict عمومی کاربر از <code>session</code> .
---------------------------	---

مدل ORM و عملیات CRUD حساب‌های ورود برنامه (app_users)؛ کاملاً مستقل از dim_customer تحلیلی در db_mock.py.

- گذرواژه با salt، PBKDF2-HMAC-SHA256 تصادفی ۱۶ بایتی و ۱۲۰ هزار iteration به صورت `pbkdf2_sha256$120000$salt$hash` ذخیره می‌شود.
- مقایسه digest با `hmac.compare_digest` (مقاوم در برابر timing attack) انجام می‌شود.
- مدیر bootstrap از environment یا مقادیر پیش‌فرض ساخته می‌شود و در هر startup فعال/admin نگه داشته می‌شود.
- کد صریحاً از حذف، غیرفعال‌سازی یا تنزل آخرین مدیر فعال جلوگیری می‌کند (بررسی `admin_count <= 1` در سه تابع).

AppUser کلاس ORM جدول، password_hash، username، id، app_users (display_name، role، is_active، created_at، updated_at)

hash_password /
verify_password هاش و بررسی گذرواژه.

init_users_table /
seed_bootstrap_admin راه‌اندازی اولیه جدول و حساب مدیر.

authenticate_user بررسی وجود، فعال بودن و صحت رمز.

list_users / create_user /
update_user / delete_user عملیات مدیریت با کنترل‌های کسب‌وکار.

حافظه گفتگو و ترجیحات جداگانه برای هر username، با persistence در history.json و کلاس

MemoryManager

- ساختار دیسک: `{"users": {username: {"preferences": {...}, "messages": [...]}}}`
- یک `threading.Lock` از تداخل نوشتن/خواندن همزمان چند درخواست جلوگیری می‌کند.
- هر پیام موفق اکنون شامل کلید `data` (نمونه ردیف‌های واقعی نتیجه، حداکثر ۵۰۰ تا) است — این افزوده تازه است که «بازپخش» کامل یک گفتگوی قدیمی (جدول + نمودار + تحلیل) را از سمت فرانت‌اند ممکن می‌کند.
- `get_frequent_questions` (تازه) روی همه کاربران سؤالات را نرمال (lower-case) و شمارش می‌کند، «SQL run» پیش‌فرض را نادیده می‌گیرد و بر اساس (تعداد، جدیدترین) مرتب می‌کند.
- فرمت قدیمی که یک list ساده بوده به bucket کاربر anonymous مهاجرت داده می‌شود؛ history.json در Git نیست.

MemoryManager کلاس اصلی با قفل نخ، بارگذاری/ذخیره دیسک و همه متدهای زیر.

_ensure_user ساخت bucket و تکمیل preference‌های پیش‌فرض.

add_message / get_context ثبت پیام (به‌همراه data) و دریافت n پیام پایانی.

get_frequent_questions(n=10) شمارش سؤالات میان همه کاربران، مرتب‌شده بر اساس تکرار و تازگی.

update_last_analysis افزودن تحلیل به آخرین سؤال مطابق.

set_feedback / _append_feedback_log ثبت 👍 / 🗨 روی پیام و لاگ جداگانه در feedback.json برای ارزیابی.

get_preferences / set_preferences مدیریت provider و language هر کاربر.

لایهٔ اصلاح خطاهای رایج مدل (regex-based، نه parser کامل SQL) و رد SQLهای غیرخواندنی پیش از اجرا.

- `fix_date_key_sql` عبارت‌های اشتباه روی `fact_transactions.date_key` (عدد YYYYYMMDD) را تشخیص و در صورت نیاز خودکار `JOIN dim_date` اضافه و ارجاع را به `full_date` بازنویسی می‌کند.
- `fix_sqlite_date_modifiers` عبارت نامعتبر «first/last day of month» SQLite را به «start of month» معتبر تبدیل می‌کند.
- `fix_status_literals` بزرگی/کوچکی حروف رایج غلط (Active/Inactive) را نرمال می‌کند.
- `strip_unrequested_limit` فقط وقتی کاربر عدد مشخصی نخواست (فارسی یا انگلیسی)، LIMIT اضافه‌شدهٔ مدل را حذف می‌کند.
- `validate_select_only` دروازهٔ نهایی است: فقط یک SELECT/WITH بدون semicolon میانی، بدون کلیدواژهٔ مخرب (.../DROP/DELETE/UPDATE) و LIMIT حداکثر ۱۰۰ هزار.

`fix_date_key_sql` بازنویسی تاریخ عددی به join با `dim_date`.

`fix_sqlite_date_modifiers / fix_status_literals` دو اصلاح متنی هدفمند دیگر.

`normalize_generated_sql` اجرای زنجیرهٔ هر سه اصلاح روی یک SQL.

`user_requested_row_limit / strip_unrequested_limit` تشخیص درخواست تعداد مشخص و حذف LIMIT ناخواسته.

`validate_select_only` دروازهٔ نهایی SQL خواندنی؛ در صورت رد شدن `UnsafeSQLError` می‌دهد.

کلاینت HTTP مدل محلی Ollama با قرارداد عمومی مشترک با .avalai_client.py.

- به `127.0.0.1:11434` متصل می‌شود و `trust_env=False` باعث می‌شود proxy سیستم در Windows در اتصال loopback دخالت نکند.
- خروجی SQL باید JSON با کلیدهای `sql` و `explanation` باشد؛ زنجیره `_extract_json` خروجی خام مدل (fence مارک‌داون، quote تکی، comma پایانی، newline خام داخل رشته) را تحمل می‌کند.
- `fix_sql` (تازه) تابع سوم قرارداد است: SQL شکست‌خورده + متن دقیق خطای دیتابیس را می‌گیرد و همان قرارداد JSON اصلاح‌شده را برمی‌گرداند — برای حلقه خوداصلاحی در `app.py`.
- تولید تحلیل متن آزاد با `temperature` بالاتر (۰.۲) و زبان فارسی/انگلیسی قابل انتخاب انجام می‌شود.

`_post` فراخوانی `api/generate/` و تبدیل خطاهای شبکه به `OllamaError`.

تعمیر `newline/tab` غیرمجاز داخل رشته JSON مدل. `_escape_literal_control_chars`
`_in_strings`

زنجیره استخراج مقاوم JSON از خروجی خام. `_try_parse / _extract_json`

ساخت `prompt` دقیق SQLite (قواعد تاریخ/وضعیت) و بازگرداندن `dict`. `generate_sql`

ساخت `prompt` اصلاح خطا و فراخوانی مدل برای SQL تصحیح‌شده (تازه). `_fix_sql_prompt / fix_sql`

تحلیل نمونه داده و بحث چندنوبتی. `generate_analysis /`
`generate_discussion`

کلاینت API سازگار با OpenAI برای AvalAI، با همان interface سه‌تایی ماژول Ollama.

- مدل از `AVALAI_MODEL` (پیش‌فرض gpt-4o-mini) و کلید از `AVALAI_API_KEY` خوانده می‌شود.
- **نکته امنیتی:** مقدار پیش‌فرض `AVALAI_API_KEY` یک کلید واقعی‌نما مستقیم در سورس است (در این سند بازنشر نشده). باید فوراً `revoke/rotate` و `default` خالی شود.
- `fix_sql` (تازه) از همان `_fix_sql_prompt` مشترک با `ollama_client.py` استفاده می‌کند تا رفتار دو provider در حلقه خوداصلاحی یکسان بماند.
- `check_credit` به endpoint حساب AvalAI متصل می‌شود؛ کلید هرگز مستقیماً به مرورگر ارسال نمی‌شود (پروکسی از طریق `app.py`).

`_chat` فراخوانی `chat/completions` و مدیریت خطاهای

`.Connection/Timeout/HTTP/Format`

`check_credit` دریافت اعتبار حساب از سمت سرور.

سه تابع اصلی، هم‌قرارداد با Ollama.
`generate_sql /`
`generate_analysis /`
`generate_discussion`

`fix_sql` اصلاح SQL شکست‌خورده با AvalAI (تازه).

قواعد سیستمی و سازنده prompt مشترک بین دو provider برای تحلیل و بحث — تنها یک‌بار نوشته و در هر دو کلاینت import می‌شود.

- قواعد صحت (accuracy rules) صریحاً از «روند نزولی مداوم» ادعاکردن جلوگیری می‌کند مگر واقعاً هر دوره پیاپی کمتر از قبلی باشد.
- برای بحث/چالش، در صورت اصلاح یک بخش مشخص، مدل باید پاسخ را با بلوک `CORRECTED_SNIPPET:` ببندد تا فرانت‌اند بتواند جایگزینی خودکار انجام دهد.

دو رشته ثابت قوانین سیستمی.
`ANALYSIS_SYSTEM_RULES /`
`DISCUSSION_SYSTEM_RULES`

ترکیب داده و سؤال برای تحلیل.
`build_analysis_prompt`

ترکیب تحلیل فعلی، تمرکز کاربر و تاریخچه بحث.
`build_discussion_prompt`

نگاشت نام ستون‌های انگلیسی دیتابیس/SQL به عنوان فارسی خوانا؛ فقط برای نمایش UI و هدر خروجی‌ها — خود دیتابیس انگلیسی می‌ماند.

- یک دیکشنری دقیق (COLUMN_LABELS) برای نام‌های شناخته‌شده و یک نگاشت کلمه‌به‌کلمه (_WORD_FA) برای snake_case/camelCase ناشناخته دارد.
- اگر ستون از قبل فارسی باشد (یا شامل نیم‌فاصله باشد) بدون تغییر برگردانده می‌شود.
- هم در جدول/نمودار static/script.js (از طریق منطق مشابه سمت کلاینت) و هم در pdf_generator.py و excel_generator.py استفاده می‌شود.

column_label(key) تابع اصلی نگاشت؛ دقیق → کلمه‌به‌کلمه → بازنگشت به متن اصلی.

is_rtl_text تشخیص متن فارسی/عربی برای تصمیم جهت.

خروجی PDF فارسی/راست‌به‌چپ برای نتیجه گفتگو، با کلاس PDFGenerator مبتنی بر fpdf2 و شکل‌دهی متن HarfBuzz.

- `_resolve_unicode_font` اولین فونت یونیکد موجود را از میان Noto یا Vazirmatn، Tahoma، Segoe UI، Arial یا Sans Arabic انتخاب می‌کند؛ در نبود هیچ‌کدام خطا می‌دهد.
- `normalize_persian_pdf_text` نیم‌فاصله (ZWNJ) را با یک فاصله نازک جایگزین می‌کند چون fpdf2+HarfBuzz آن را حذف و حروف را می‌چسباند.
- `set_text_shaping(True)` فعال است تا حروف فارسی به‌درستی به هم متصل شوند (چیزی که بدون HarfBuzz در PDF کار نمی‌کند).
- جهت هر سلول به‌صورت پویا بر اساس RTL بودن متن آن سلول تعیین می‌شود، نه یک تنظیم سراسری ثابت.
- صفحه شامل نوار عنوان، توضیح، تصویر نمودار (در صورت وجود)، جدول داده (حداکثر ۸۰ ردیف با یادداشت برش) و تحلیل است؛ هدر/فوتر تکرارشونده از طریق override متدهای `header/footer` پیاده شده.

PDFGenerator زیرکلاس FPDF با هدر/فوتر برنندشده و متد `section_title`.

`_resolve_unicode_font` انتخاب فونت یونیکد در دسترس سیستم.

`normalize_persian_pdf_text` رفع مشکل حذف نیم‌فاصله در رندر.

`format_number` قالب‌بندی عدد با جداکننده هزارگان قابل تنظیم.

`generate_llm_pdf` تابع ورودی اصلی: عنوان، توضیح، داده، تحلیل، تصویر نمودار → فایل PDF.

خروجی اکسل سه‌شیتی (Data / Explanation / Analysis) با استایل‌دهی openpyxl.

- شیت Data شامل هدر رنگی، فریز ردیف اول، فیلتر خودکار، رنگ‌آمیزی ردیف زوج و راست‌چین کردن خودکار متن فارسی است.
- اعداد رشته‌ای (با کاما/سمی‌کالن) قبل از نوشتن به نوع عددی واقعی اکسل تبدیل می‌شوند تا SUM/فیلتر عددی در اکسل درست کار کند.
- پهناي ستون‌ها با `_autosize` بر اساس طولانی‌ترین مقدار هر ستون تنظیم می‌شود.

ورودی اصلی: سه شیت را می‌سازد و فایل را ذخیره می‌کند. `generate_llm_excel`

تشخیص و تبدیل مقدار متنی به عدد واقعی. `_is_number / _to_number`

تنظیم خودکار عرض ستون. `_autosize`

نقطه ورود میزبانی cPanel/Phusion Passenger؛ مسیر استقرار جایگزین Dockerfile.

- دایرکتوری برنامه را به `sys.path` اضافه می‌کند تا import های نسبی کار کنند.
- چون FastAPI اپلیکیشن ASGI است ولی Passenger انتظار WSGI دارد، `a2wsgi.ASGIMiddleware` پل بین این دو را می‌زند.

شیء `application` entry point مورد انتظار Passenger.

فایل‌های Frontend (HTML/CSS/JavaScript)

خط 717

static/first.html

صفحه عمومی معرفی Rayamate که مسیر ریشه سایت (/ GET) آن را نمایش می‌دهد.

- Tailwind از CDN، آیکون Iconify و فونت Vazirmatn از گوگل بارگذاری می‌شود؛ CSS تکمیلی inline است.
- بخش‌های hero، آمار متحرک، نمایش محصول، قابلیت‌ها، CTA و فوتر شامل لینک شبکه‌های اجتماعی دارد؛ لینک لینکدین به آدرس رسمی صفحه شرکت (linkedin.com/company/rayamate) تنظیم شده است.
- Canvas ذرات متحرک و خطوط ارتباطی پس‌زمینه، شمارنده آماری با IntersectionObserver و افکت tilt کارت‌ها با جاوااسکریپت inline پیاده‌سازی شده‌اند.

خط 634

static/login.html

صفحه ورود راست‌به‌چپ با تم سبز CSS، Rayamate کامل inline و اتصال به login.js.

- طرح دو ستونه (معرفی محصول + فرم ورود)؛ در موبایل تک‌ستونه می‌شود.
- ورودی نام کاربری، رمز، «مرا به خاطر بسپار»، چک‌باکس «من ربات نیستم» (کپچای نمایشی، نه واقعی)، دکمه نمایش/پنهان رمز و ناحیه پیام خطا دارد.

کنترل رفتار فرم ورود در مرورگر.

- فقط username را (در صورت فعال بودن «مرا به خاطر بسپار») در localStorage نگه می‌دارد و هرگز password را ذخیره نمی‌کند.
- قبل از ارسال، خالی نبودن فیلدها و تیک‌خوردن چک‌باکس کپچا را بررسی می‌کند.
- درخواست JSON به `/api/login` می‌فرستد و پس از موفقیت به `/app` هدایت می‌شود؛ دکمه‌های «بازیابی رمز» و «ورود سازمانی» فعلاً فقط پیام «به‌زودی» نشان می‌دهند.

`submit handler` اعتبارسنجی فرم، غیرفعال‌سازی موقت دکمه، مدیریت پاسخ/خطا.

`toggle password handler` تغییر type بین password و text.

`showError` نمایش پیام خطای فارسی.

پوسته داشبورد احرازشده؛ فایل واقعی سروشده روی مسیر `/app`.

- هدر شامل عنوان، دکمه تعویض زبان، لینک مدیریت کاربران (فقط برای admin) و دکمه خروج است.
- ستون گفتگو شامل پیام خوش‌آمد، ناحیه چت، ورودی متن و دکمه‌های صدا/PDF/Excel/ارسال است.
- نوار کناری (sidebar) اکنون پنج کارت دارد: «تاریخچه گفتگو» (قابل کلیک برای بازپخش کامل)، «۱۰ سؤال پرتکرار» (تازه — سراسر کاربران)، «اقدامات سریع»، «تنظیمات» و «نظرسنجی قابلیت‌های آینده»؛ عرض نوار کناری با drag قابل تغییر و قابل جمع‌شدن است.
- Chart.js محلی (vendored) و static/script.js در پایان صفحه بارگذاری می‌شوند.

منطق کامل کلاینت داشبورد؛ بدون هیچ فریم‌ورک، مبتنی بر DOM API خام (بزرگ‌ترین فایل پروژه).

- در startup از `/api/preferences` و `/api/me` ترجیحات و نقش کاربر را می‌خواند، تاریخچه و سؤالات پرتکرار را بارگذاری می‌کند.
- هر «نوبت» گفتگو (turn) از یک SQL قابل‌ویرایش، سپس نتایج (اکنون در کارت‌های رنگی مجزا) تشکیل می‌شود: کارت SQL (آبی)، کارت جدول (سبز)، کارت نمودار (بنفش)، کارت تحلیل (گرادیان سبز/نارنجی موجود قبلی)، کارت بحث (کهربایی، اکنون به‌صورت آکاردئون جمع‌شونده) و کارت بازخورد (خاکستری).
- کلیک روی یک آیتم تاریخچه اکنون کل نوبت قدیمی را «بازپخش» می‌کند (سؤال، SQL، جدول، نمودار، تحلیل، وضعیت بازخورد قبلی) به‌جای فقط پرکردن کادر ورودی؛ برای پیام‌های قدیمی‌تر که داده ذخیره‌شده ندارند، پیام راهنما نشان داده می‌شود.
- برای نمودار، ستون عددی/برچسب هوشمند انتخاب می‌شود و انواع `bar/barHorizontal/line/pie/doughnut` پشتیبانی می‌شوند؛ تعویض نوع نمودار اکنون از یک تابع واحد `rebuildChart` عبور می‌کند (پیش‌تر سه بلوک کد تکراری بود که باعث باگ سویچ‌نکردن نمودار می‌شد).
- یک دکمه «چاپ نمودار» (تازه) نمودار جاری را به PNG تبدیل و در یک `iframe` مخفی برای چاپ/ذخیره PDF باز می‌کند.
- تحلیل مدل `escape` و سپس `heading/list` عددی‌ها به HTML کنترل‌شده تبدیل می‌شود تا XSS ممکن نباشد.

state	شیء وضعیت سراسری: زبان، provider، آخرین پاسخ، آخرین نمودار.
apiFetch / logout / savePreferences	ارتباط session-aware با backend.
loadHistorySidebar / replayHistoryEntry	نمایش تاریخچه و بازپخش کامل یک نوبت قدیمی (تازه).
loadFrequentQuestions	بارگذاری و نمایش ۱۰ سؤال پرتکرار سراسری (تازه).
sendMessage / renderSqlDraft / renderRunResults	چرخه کامل هر نوبت گفتگو، اکنون در کارت‌های مجزا.
wireSqlRunner	اجرای SQL، نمایش نتیجه یا خطا، و یادداشت «اصلاح خودکار» در صورت auto_fixed (تازه).
prepareChartRows / createChartInstance / rebuildChart	ساخت و تعویض ایمن نمودار Chart.js (بازنگری‌شده).
printChartAsImage	خروجی چاپی نمودار به‌صورت تصویر (تازه).

منطق بحث چندنوبتی + تاگل باز/بسته‌شدن آکاردئون (تازه).

wireDiscussPanel

ساخت امن و صفحه‌بندی جدول نتایج.

mountPaginatedTable /
buildTableHtml

ایمنی خروجی و پشتیبانی دوجته.

escapeHtml /
formatAnalysisHtml /
isRtlText

خط 1,953

static/style.css

سامانه طراحی مشترک داشبورد و صفحه کاربران؛ الهام از فیش تسویه POS (ledger/receipt).

- متغیرهای رنگ، کاغذ، danger، accent، فونت sans/mono/RTL را در root: تعریف می‌کند.
- هر «نوبت» گفتگو اکنون از چند کارت رنگی مجزا (به‌جای یک نوار پیوسته با خطچین) تشکیل شده: هر بخش (feedback-section، discuss-section، chart-section، table-section، sql-draft-section) یک نوار رنگی بالای خودش دارد که با متغیر --section-accent تعیین می‌شود.
- بخش «بحث و بررسی» اکنون آکاردئون است: discuss-body با کلاس is-open روی discuss-section کنترل می‌شود.
- استایل جدول pagination، نمودار، تحلیل، نوار زمان‌بندی، حالت loading و breakpoint‌های واکنش‌گرا را نیز دارد.

خط 210

static/users.html

صفحه CRUD کاربران، فقط برای نقش مدیر.

- استایل اختصاصی صفحه inline است و style.css مشترک جداگانه بارگذاری می‌شود.
- جدول کاربران، حالت خالی، پیام عملیات و modal افزودن/ویرایش دارد؛ فرم شامل نام نمایشی، username، موبایل، role، password و وضعیت فعال است.

اتصال UI مدیریت کاربران به API های admin در app.py.

- wrapper مشترک `api()` پاسخ 401 را به `login/` و پاسخ 403 را به `app/` هدایت می‌کند.
- در ساخت کاربر جدید رمز الزامی است؛ در ویرایش اختیاری (خالی = بدون تغییر رمز).
- پیش از بارگذاری فهرست کاربران، ابتدا `/api/me` نقش را کنترل می‌کند تا غیرمدیر به `app/` هدایت شود.

`api` fetch با تبدیل پاسخ خطا/403/401 به رفتار مناسب.

`openModal / closeModal` حالت `create/edit` فرم.

`renderUsers / loadUsers` رندر جدول و دریافت فهرست.

`removeUser` تأیید و درخواست `DELETE`.

نسخه `minified` و `vendored` کتابخانه `Chart.js v4` برای رسم نمودار بدون وابستگی runtime به CDN.

- کد `third-party` تولیدشده است؛ نگهداری دستی آن توصیه نمی‌شود.
- از طریق `global` به نام `Chart` در `static/script.js` مصرف می‌شود.

فایل‌های پیکربندی، مستندسازی و legacy

Dockerfile

خط 17

ساخت image سبک Python 3.11 برای اجرا در container یا Hugging Face Spaces.

- dependencyها پیش از COPY کل پروژه نصب می‌شوند تا cache لایه‌ها بهتر استفاده شود.
- پورت ۷۸۶۰ expose می‌شود؛ در startup اگر فایل DB وجود نداشته باشد db_mock.py اجرا و سپس Uvicorn بالا می‌آید.

requirements.txt

خط 11

فهرست dependencyهای runtime پایتون.

- FastAPI/Uvicorn برای وب، SQLAlchemy/Pandas برای داده، Requests برای دو کلاینت مدل، python-multipart/itsdangerous برای multipart، a2wsgi برای session، fpdf2/uharfbuzz برای PDF فارسی و openpyxl برای اکسل.
- برخی packageها pin نشده‌اند (مثل pandas و requests)؛ برای build تکرارپذیر بهتر است نسخه ثابت شود.

README.md

خط 178

راهنمای اولیه پروژه؛ اکنون با کد فعلی به‌روز نیست.

- همچنان معماری access_key/dim_customer و endpointهایی مثل /api/customers را توصیف می‌کند، در حالی که برنامه فعلی احراز هویت session-based با جدول app_users و مجموعه endpoint متفاوتی دارد.
- بخش‌های نصب Ollama، ساخت DB و اجرای Uvicorn همچنان درست و مفیدند.

حذف فایل‌های پایتون، محیط مجازی، DB/history/env، آرشیو و تنظیمات IDE از Git.

- به‌درستی `history.json`، `*.db`، `.env` و `feedback.json` را نادیده می‌گیرد.
- الگوی `* copy.py` باعث نادیده‌گرفتن فایل‌های پشتیبان دستی مثل `ollama_client copy.py` می‌شود؛ کلید/رمز نوشته‌شده مستقیم در سورس (مثل `avalai_client.py`) با `gitignore` محافظت نمی‌شود.

فایل داده‌ ساده (نه کد) که آرای نظرسنجی قابلیت‌های آینده را نگه می‌دارد؛ توسط `app.py` خوانده/نوشته می‌شود.

- ساختار: `{"votes": {username: [option_id, ...]}}`؛ گزینه‌های سفارشی با پیشوند `_custom_` ذخیره می‌شوند.

اسکرپت مستقل ساخت ارائه فارسی PowerPoint دربارهٔ پروژه (ابزار مستندسازی، نه بخشی از اپلیکیشن).

- با `python-pptx` شکل‌ها، متن‌باکس‌ها، بولت‌ها و اسلایدهای فارسی راست‌به‌چپ می‌سازد.
- `python-pptx` dependency در `requirements.txt` نیست؛ برای بازتولید ارائه باید جداگانه نصب شود.

نسخهٔ قبلی اسکرپت ساخت ارائه، مخصوص یک برنچ قدیمی‌تر (`newDashboard2`)؛ ابزار مستندسازی است، بخشی از اپلیکیشن نیست.

- امروز جایگزین بهتری برای این اسکرپت لازم نیست؛ فقط برای بازتولید تاریخی نگه داشته شده.

مولد HTML/PDF مستندات فنی فارسی برای برنچ قدیمی newDashboard2 (با `git show` از یک `commit` خاص)؛ الگوی طراحی همین سند از آن اقتباس شده است.

- خروجی‌های آن (`newDashboard2_code_documentation_fa.html/.pdf`) در Git باقی مانده‌اند؛ محتوایشان مربوط به وضعیت قدیمی‌تر کد است، نه نسخه فعلی.
- این سند («`bichart_project_documentation_fa.pdf`») بر خلاف آن، مستقیماً از `working tree` فعلی (شامل تغییرات ثبت‌نشده) ساخته می‌شود، نه از یک `commit` گیت.

نسخه قدیمی/مرجع پوسته گفتگو در ریشه FastAPI — repository این فایل را سرو نمی‌کند.

- مسیر فعال به `static/index.html` اشاره دارد؛ این فایل احتمالاً باقی‌مانده طراحی قبلی است.

جاوااسکریپت قدیمی رابط `access_key` در ریشه؛ در صفحات فعال بارگذاری نمی‌شود.

- DOM‌هایی مثل `login-gate` و API‌هایی مثل `api/customers/` را انتظار دارد که با `app.py` فعلی هماهنگ نیستند.

CSS قدیمی رابط تک‌ستونه در ریشه؛ توسط `FastAPI mount` نشده و صفحات فعال آن را نمی‌خوانند.

- توکن‌های هویت بصری اولیه و `login-gate` قدیمی را نگه می‌دارد.

فایل‌های اضافی/موقتی که در **working tree** هستند ولی توسط اپلیکیشن استفاده نمی‌شوند:

- `ollama_client copy.py` — نسخه پشتیبان دستی، توسط `gitignore` نادیده گرفته می‌شود و هیچ‌جا `import` نشده است.
- `change_logo_color.py` و `test_pdf.py` — اسکریپت‌های یک‌بارمصرف توسعه (تغییر رنگ لوگو با `Pillow`، تست دستی `pdf_generator.py`)؛ بخشی از مسیر اجرای برنامه نیستند.
- ~۹۰ فایل `test_font_*.pdf`، `report*.pdf/.xlsx` و مشابه در ریشه پروژه — خروجی‌های تولیدشده هنگام آزمایش `export_pdf/export_excel` و انتخاب فونت؛ می‌توانند با خیال راحت حذف شوند.
- `Rayamate_Presentation.pptx` و `newDashboard2_code_documentation_fa.html/.pdf` — خروجی‌های ابزارهای بالا، نه سورس کد.

جریان‌های اجرایی مهم

ورود

login.js → POST /api/login → users.authenticate_user → verify_password → set_session_user → cookie session → redirect /app

گفتگو + اجرا + خوداصلاحی

sendMessage → POST /api/chat → provider.generate_sql → wireSqlRunner → POST /api/run_sql → normalize/validate → execute → (در صورت خطا) provider.fix_sql → validate → execute → مجدد → memory.add_message(data=...) → renderRunResults

تحلیل

تحلیل دکمه → POST /api/analyze → provider.generate_analysis(نمونه ۸۰ ردیف) → memory.update_last_analysis → formatAnalysisHtml

بحث

discuss-composer (داخل آکاردئون) → POST /api/discuss → provider.generate_discussion(تاریخچه) → appendBubble (محدود)

تاریخچه و بازپخش

بازسازی کامل جدول/نمودار/ → (ذخیره‌شده data با entry) replayHistoryEntry → کلیک آیتم تاریخچه → تحلیل بدون فراخوانی مجدد مدل

سؤالات پرتکرار

loadFrequentQuestions → GET /api/frequent-questions → memory.get_frequent_questions
کلیک = ارسال فوری سؤال → (شمارش سراسری)

خروجی PDF/Excel

دکمه PDF/Excel → POST /api/export_pdf یا /api/export_excel →
pdf_generator/excel_generator → دانلود فایل

مدیریت کاربر

/users → require_admin → users.js CRUD → /api/users → users.py → SQLAlchemy app_users

ریسک‌ها و پیشنهادهای فنی

بالا — credentialهای پیش‌فرض

نام کاربری/گذرواژه مدیر (bootstrap (users.py و SESSION_SECRET (auth.py مقدار پیش‌فرض ثابت دارند. در production باید بدون environment امن، startup fail شود.

بحرانی — کلید API در سورس

در avalai_client.py برای AvalAI یک مقدار پیش‌فرض واقعی‌نما مستقیم در کد وجود دارد (در این سند بازنشر نشده). باید فوراً revoke/rotate شود و فقط از environment/secret manager خوانده شود؛ مقدار پیش‌فرض باید خالی باشد.

متوسط — CSRF و rate limit

عملیات POST/PUT/DELETE توکن ضد-CSRF ندارند و هیچ محدودیت نرخ درخواستی روی api/run_sql/ یا api/chat/ وجود ندارد، در حالی که این مسیرها به مصرف پولی AvalAI منجر می‌شوند.

متوسط — کپچای نمایشی

کپچا فقط یک boolean ارسالی از مرورگر است، نه اثبات ضدریات واقعی. برای دسترسی عمومی باید سرویس captcha واقعی و rate limiting اضافه شود.

متوسط — نبود row-level scope واقعی

SCHEMA_DESCRIPTION مدل را راهنمایی می‌کند اما هیچ محدودیت اجباری در لایه SQL برای محدودکردن نتیجه به مشتری/پذیرنده مشخص کاربر وجود ندارد؛ app_users هم به dim_customer متصل نیست.

متوسط — اعتبارسنجی SQL بر پایه regex

sql_safety.py برای نمونه اولیه مناسب است ولی parser کامل SQL نیست؛ اجازه WITH/SELECT، نبود timeout یا محدودیت هزینه query، و امکان کوئری سنگین باقی می‌ماند. کاربر DB باید read-only باشد.

پایین — dependency بدون pin و CDN

برخی packageها نسخه ثابت ندارند و first.html/login.html به Tailwind/فونت CDN وابسته‌اند؛ برای شبکه بسته باید asset محلی شود.

پایین — persistence تک‌پردازهای

قفل MemoryManager فقط داخل یک process معتبر است؛ چند worker می‌توانند history.json را همزمان بازنویسی کنند.

راه اندازی و استقرار

1. **محیط:** Python 3.11 و virtualenv بسازید؛ سپس `pip install -r requirements.txt`.
2. **تنظیم secrets:** حداقل `SESSION_SECRET` ، `AUTH_USERNAME` ، `AUTH_PASSWORD` و در صورت استفاده از AvalAI مقدار `AVALAI_API_KEY` را در محیط امن قرار دهید (نه در سورس).
3. **مدل محلی (اختیاری):** برای Ollama مدل تنظیم شده در `(ollama_client.py)` را `MODEL_NAME` را `pull` و `ollama serve` را اجرا کنید.
4. **داده mock:** یکبار `python db_mock.py` — توجه: اجرای مجدد دیتابیس را بازسازی می کند.
5. **اجرا:** `uvicorn app:app --host 0.0.0.0 --port 8000` سپس بازکردن مسیر ریشه در مرورگر.
6. **استقرار:** Dockerfile برای container/Hugging Face Spaces و `passenger_wsgi.py` برای cPanel دو مسیر مستقل استقرارند.

فهرست کامل فایل‌های پروژه

ردیف	مسیر	اندازه
1	.gitignore	29 خط
2	.test_cookies.txt	4 خط
3	Dockerfile	17 خط
4	README.md	178 خط
5	Rayamate_Presentation.pptx	46,614 بایت
6	analysis_prompt.py	69 خط
7	app.py	935 خط
8	auth.py	56 خط
9	avalai_client.py	196 خط
10	change_logo_color.py	63 خط
11	column_labels.py	203 خط
12	create_presentation.py	436 خط
13	database.py	52 خط
14	db_mock.py	412 خط
15	excel_generator.py	129 خط
16	feature_poll.json	9 خط
17	feedback.json	43 خط
18	generate_newDashboard2_docs.py	720 خط
19	generate_newDashboard2_slides.py	423 خط
20	generate_project_docs_fa.py	899 خط
21	history.json	7,269 خط

ردیف	مسیر	اندازه
22	index.html	73 خط
23	memory_manager.py	238 خط
24	newDashboard2_code_documentation_fa.html	537 خط
25	newDashboard2_code_documentation_fa.pdf	526,978 بایت
26	newDashboard2_code_slides_fa.html	271 خط
27	newDashboard2_code_slides_fa.pdf	229,979 بایت
28	ollama_client copy.py	167 خط
29	ollama_client.py	315 خط
30	passenger_wsgi.py	20 خط
31	pdf_generator.py	336 خط
32	psp_bi_mock.db	188,416 بایت
33	report20260727092959.pdf	98,314 بایت
34	report20260727093000.xlsx	9,209 بایت
35	report20260727093002.pdf	98,314 بایت
36	report20260727093003.pdf	98,314 بایت
37	report20260727100429.xlsx	7,377 بایت
38	report20260727100430.pdf	49,939 بایت
39	report20260727100435.pdf	49,939 بایت
40	report20260727122231.pdf	95,416 بایت
41	report20260727122233.pdf	95,416 بایت
42	report20260727122234.pdf	95,416 بایت
43	report20260727132829.pdf	87,286 بایت
44	report20260727132837.pdf	87,286 بایت
45	report20260727132839.pdf	87,286 بایت

ردیف	مسیر	اندازه
46	report20260727132840.pdf	87,286 بایت
47	report20260727141539.pdf	97,383 بایت
48	report20260727141541.pdf	97,383 بایت
49	report20260727141542.pdf	97,383 بایت
50	report20260727153803.pdf	92,754 بایت
51	report20260727153805.pdf	92,754 بایت
52	report20260727162351.pdf	42,047 بایت
53	report20260727162353.pdf	42,047 بایت
54	report_1785072381.pdf	13,957 بایت
55	report_1785072397.pdf	13,957 بایت
56	report_1785078169.pdf	30,999 بایت
57	report_1785078174.pdf	30,999 بایت
58	report_1785078610.pdf	34,872 بایت
59	report_1785078613.pdf	34,872 بایت
60	report_1785078640.pdf	43,111 بایت
61	report_1785078642.pdf	43,111 بایت
62	report_1785079116.pdf	41,831 بایت
63	report_1785079118.pdf	41,831 بایت
64	report_1785079512.xlsx	9,327 بایت
65	report_1785079566.pdf	105,212 بایت
66	report_1785079569.pdf	105,212 بایت
67	report_1785079570.pdf	105,212 بایت
68	report_1785079842.xlsx	7,311 بایت
69	report_1785079941.pdf	79,515 بایت

ردیف	مسیر	اندازه
70	report_1785079943.pdf	79,515 بایت
71	report_1785080377.xlsx	9,326 بایت
72	report_1785080378.pdf	99,408 بایت
73	report_1785080380.pdf	99,408 بایت
74	report_1785080401.xlsx	9,326 بایت
75	report_1785081011.pdf	99,651 بایت
76	report_1785081013.pdf	99,651 بایت
77	report_1785083104.pdf	91,836 بایت
78	report_1785083119.pdf	91,836 بایت
79	report_1785083130.pdf	91,836 بایت
80	requirements.txt	11 خط
81	script.js	351 خط
82	sql_safety.py	233 خط
83	static/favicon-128.png	19,188 بایت
84	static/favicon-16.png	926 بایت
85	static/favicon-192.png	41,642 بایت
86	static/favicon-32.png	2,977 بایت
87	static/favicon-48.png	4,395 بایت
88	static/favicon-512.png	183,269 بایت
89	static/favicon-64.png	8,054 بایت
90	static/favicon.png	183,269 بایت
91	static/first.html	717 خط
92	static/fonts/NotoSansArabic-Regular.ttf	240,456 بایت
93	static/images/login-template.png	74,539 بایت

ردیف	مسیر	اندازه
94	static/index.html	130 خط
95	static/login.html	634 خط
96	static/login.js	101 خط
97	static/logo.png	183,269 بایت
98	static/script.js	2,382 خط
99	static/style.css	1,953 خط
100	static/users.html	210 خط
101	static/users.js	179 خط
102	static/vendor/chart.umd.min.js	20 خط
103	style.css	426 خط
104	test_font_NotoSansArabic-Regular_False.pdf	7,558 بایت
105	test_font_NotoSansArabic-Regular_True.pdf	7,605 بایت
106	test_font_arial_False.pdf	12,278 بایت
107	test_font_arial_True.pdf	12,880 بایت
108	test_font_segoeui_False.pdf	10,239 بایت
109	test_font_segoeui_True.pdf	10,530 بایت
110	test_font_tahoma_False.pdf	11,054 بایت
111	test_font_tahoma_True.pdf	11,603 بایت
112	test_output.pdf	20,464 بایت
113	test_output.xlsx	6,319 بایت
114	test_pdf.py	21 خط
115	test_persian.pdf	20,323 بایت
116	test_styled.pdf	20,093 بایت
117	test_styled.xlsx	6,462 بایت

ردیف	مسیر	اندازه
118	test_zwnj.pdf	18,495 بایت
119	test_zwnj_0.pdf	16,827 بایت
120	test_zwnj_1.pdf	16,827 بایت
121	test_zwnj_2.pdf	16,289 بایت
122	test_zwnj_fixed.pdf	17,704 بایت
123	users.py	245 خط

جمع‌بندی

bichart یک ابزار BI مکالمه‌ای نسبتاً کامل است: صفحه معرفی، ورود، داشبورد گفتگو با تاریخچه قابل‌بازپخش و سؤالات پرتکرار سراسری، مدیریت کاربران، انتخاب دو مدل زبانی، تولید/اصلاح/اجرای SQL با حلقه خوداصلاحی، جدول و نمودار (اکنون در کارتهای مجزا با قابلیت چاپ)، تحلیل و بحث ثانویه، و خروجی PDF/Excel فارسی را یکجا دارد.

مهم‌ترین کارهای پیش از production عبارت‌اند از: حذف/چرخش کلید API نوشته‌شده در سورس، اجباری‌کردن secret‌های امن، افزودن row-level authorization واقعی، CSRF و rate-limit، استفاده از یک دیتابیس واقعی به جای history.json برای تاریخچه، parser/timeout قوی‌تر SQL، به‌روزرسانی README و حذف فایل‌های legacy ریشه.

پایان سند • تولیدشده در {esc(generated_at)}